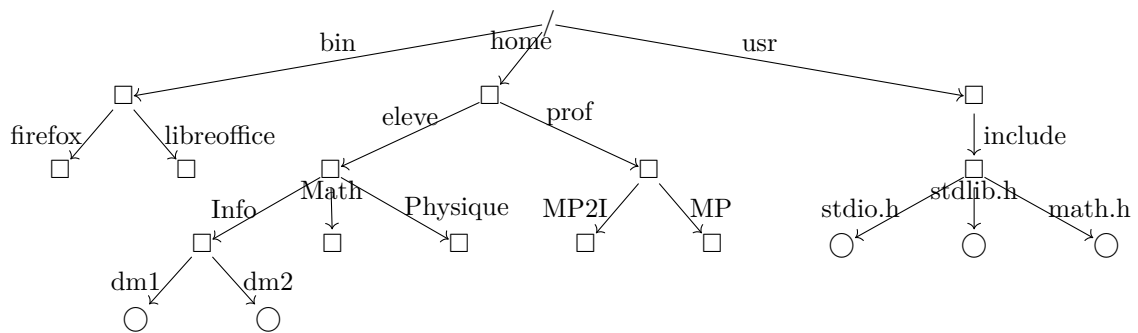


# Chapitre 1

## Système et ligne de commande

**Exercice de cours 1.1.** Quelle option de `man` permet d'avoir une liste de commandes en lien avec un mot-clef, incluant une petite description de ce que fait chaque commande de la liste ?

**Exercice de cours 1.2.** On considère l'arborescence suivante :



1. Donner un chemin absolu pour désigner le fichier `dm1`.
2. Donner un chemin absolu pour désigner le répertoire `include`.
3. Donner un chemin relatif pour désigner à partir du répertoire `usr` le fichier `math.h`.
4. Donner un chemin relatif à partir du répertoire de login de `prof` pour désigner le fichier `firefox`.

**Exercice de cours 1.3.** On considère l'arborescence de l'exercice 1.2.

1. On suppose que le répertoire courant est le répertoire `Info`. Donner une commande pour modifier le répertoire courant en la racine du système.
2. On suppose que l'utilisateur est `eleve` et que le répertoire courant est le répertoire `Info`. Donner deux commandes différentes qui permettent à l'utilisateur courant de modifier le répertoire courant en son répertoire de login.

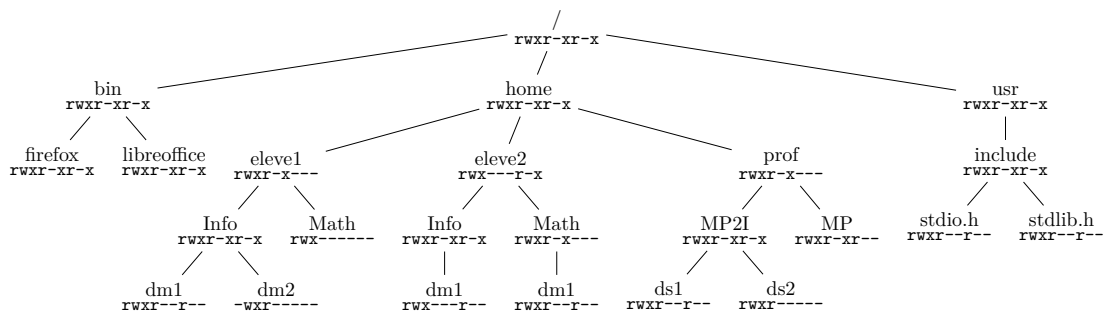
**Exercice de cours 1.4.** On considère l'arborescence de l'exercice 1.2.

1. On suppose que l'utilisateur est `eleve` et que le répertoire courant est le répertoire `Info`. Donner une commande permettant de copier le fichier `dm1` dans le répertoire `MP2I`. (À noter que dans la vraie vie il est vraisemblable que vous n'aurez pas les droits pour cela, mais ce n'est pas le problème ici.)
2. Donner une commande qui permet de supprimer le fichier `libreoffice`, quel que soit le répertoire courant. (Même remarque que pour la question précédente.)

# Chapitre 2

## Fichiers

**Exercice de cours 2.1.** On considère l'arborescence suivante avec les droits indiqués : (je mets les noms dans les nœuds et pas les arêtes, sinon ça devient illisible)



Le répertoire `prof` (resp. `eleve1`, `eleve2`) et tout ce qui est en dessous appartient à l'utilisateur `prof` (resp. `eleve1`, `eleve2`) et au groupe `enseignants` (resp. `eleves`).

Tout le reste appartient à l'utilisateur `root` et au groupe du même nom.

On considère qu'il y a 5 utilisateurs répartis sur 3 groupes :

- groupe `root` : `root` ;
- groupe `eleves` : `eleve1`, `eleve2` et `eleve3` ;
- groupe `enseignants` : `prof`

1. Quels utilisateurs ont le droit en exécution sur le fichier `/bin/firefox` ?
2. Quels utilisateurs ont le droit en lecture sur le fichier `/home/eleve1/Info/dm2` ?
3. Quels utilisateurs ont le droit en lecture sur le fichier `/home/prof/MP2I/ds1` ?
4. Quels utilisateurs ont le droit en exécution sur le fichier `/home/prof/MP` ?

**Exercice de cours 2.2.** On reprend la situation de l'exercice 2.1.

Lesquelles de ces commandes ne produisent pas d'erreur ?

1. 

```
$ whoami
eleve1
$ pwd
/home/eleve1
$ cat Info/dm1
```
2. 

```
$ whoami
eleve1
$ pwd
/home/eleve1
$ cat Info/dm2
```
3. 

```
$ whoami
eleve1
$ pwd
/home/eleve1
$ cat /home/eleve2/Math/dm1
```

4.

```
$ whoami
eleve3
$ pwd
/home
$ ../bin/libreoffice
```
5.

```
$ whoami
eleve3
$ pwd
/
$ cat prof/MP2I/ds1
```
6.

```
$ whoami
prof
$ pwd
/home/prof/MP
$ cat ../MP2I/ds2
```
7.

```
$ whoami
prof
$ pwd
/home/prof
$ rm ~eleve1/Info/dm1
```
8.

```
$ whoami
prof
$ pwd
/home/prof
$ cp ~eleve1/Info/dm1 .
```
9.

```
$ whoami
root
$ pwd
/home/usr/include
$ rm /home/eleve2/Math/dm1
```

## Chapitre 3

# Caractériser un langage de programmation

# Chapitre 4

## Le langage C : les bases

**Exercice de cours 4.1.** Quels identifiants parmi les suivants sont corrects en C ?

sevenup seven\_up 7up seven.up \_7up SevenUp

**Exercice de cours 4.2.** Dans le programme suivant, mettre en lien chaque occurrence de `n` avec la déclaration correspondante.

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int f(){
5     int n = 17;
6     if(true){
7         int p = 2 * n;
8         int n = p;
9         for(int i = 0; i < n; i = i+1){
10             int n = i + 2;
11             printf("%d ", n);
12         }
13         printf("%d ", n);
14     } else {
15         printf("%d ", -n);
16     }
17     return n;
18 }
19
20 int main(void){
21     int n = f();
22     printf("%d ", n);
23 }
```

**Exercice de cours 4.3.** Donner l'en-tête d'une fonction qui permet de calculer la valeur absolue.

**Exercice de cours 4.4.** Montrer qu'il y a  $2^8$  entiers distincts entre  $-2^7$  et  $2^7 - 1$ .

**Exercice de cours 4.5.** Donner la représentation de l'entier 42 sur 8 bits.

**Exercice de cours 4.6.** Donner la représentation en complément à 2 de  $-17$ , de  $-42$  et de  $-80$ .

**Exercice de cours 4.7.** Donner les représentations IEEE 754 des nombres suivants :  $2^{-3}$  1,5

## Chapitre 5

# Introduction à l'algorithmique

# Chapitre 6

## Correction totale

**Exercice de cours 6.1.** On considère la boucle suivante :

```
/** n est un entier positif */
int mystere(int n){
    while(n > 17){
        if(n%2 == 0){
            n = n / 2;
        } else {
            n = n - 1;
        }
    }
    return n;
}
```

Donner un variant de boucle pour la boucle `while`.

**Exercice de cours 6.2.** Montrer que tout appel à la fonction `mystere` de l'exercice 6.1 se termine.

**Exercice de cours 6.3.** On considère la fonction suivante :

```
/**
 * calcule x puissance n
 * n est un entier positif
 */
double puissance(double x, int n){
    assert(n >= 0);
    if(n == 0){
        return 1;
    }
    int y = puissance(x, n/2);
    if(n%2 == 0){
        return y * y;
    } else {
        return x * y * y;
    }
}
```

Donner un variant d'appel pour cette fonction.

**Exercice de cours 6.4.** Montrer que tout appel à la fonction `puissance` de l'exercice 6.3 se termine.

**Exercice de cours 6.5.** On considère la fonction suivante :

```
1 /** calcule m + n pour n positif */
2 int somme(int m, int n){
3     while(n != 0){
4         m = m + 1;
5         n = n - 1;
6     }
7     return m;
8 }
```

On note  $m_0$  et  $n_0$  les valeurs initiales respectives de  $m$  et  $n$ . Montrer que  $m + n = m_0 + n_0$  est un invariant pour la boucle `while`.

En déduire que la fonction est correcte.

**Exercice de cours 6.6.** On considère la fonction suivante :

```
1 /** calcule m + n pour n positif */
2 int somme(int m, int n){
3     if(n == 0){
4         return m;
5     }
6     return somme(m+1, n-1);
7 }
```

Montrer qu'elle possède une correction totale.

## Chapitre 7

# Le langage C : gestion de la mémoire

**Exercice de cours 7.1.** Donner des lignes de codes qui permettent de créer un tableau `tab` d'entiers de taille 100 dans lequel la case d'indice  $i$  contient l'entier  $2 * i$ .

**Exercice de cours 7.2.** Donner des lignes de codes qui permettent de créer un tableau `F` d'entiers de taille 100 dans lequel la case d'indice  $i$  contient le  $i^{\text{e}}$  nombre de Fibonacci défini par la suite récurrente  $F_0 = 0$ ,  $F_1 = 1$  et pour tout  $n \geq 2$ ,  $F_n = F_{n-1} + F_{n-2}$ .

**Exercice de cours 7.3.** Écrire une fonction `translation` qui prend trois paramètres : un pointeur sur entier `t`, un entier `n` correspondant au nombre de cases allouées à l'adresse `t`, et un entier `v` et ajoute la valeur `v` à chaque case de `t`.

**Exercice de cours 7.4.** Écrire une fonction `debut_N` qui prend un entier strictement positif `n` en paramètre et renvoie une adresse à laquelle `n` cases mémoire de taille `sizeof(int)` ont été allouées, représentant  $\llbracket 0, n - 1 \rrbracket$ .

**Exercice de cours 7.5.** La fonction `int atoi(const char *nptr)` prend une chaîne de caractères en argument (même type qu'un `argv[i]`) et renvoie l'entier correspondant si elle correspond à l'écriture d'un entier en base 10.

Écrire un programme qui s'arrête s'il n'a pas exactement 2 arguments, et sinon affiche la somme des deux s'ils représentent des entiers.

## Chapitre 8

# Structures en C

**Exercice de cours 8.1.** Écrire le code permettant de créer un nouveau type représentant un triangle.

**Exercice de cours 8.2.** Écrire une fonction qui prend trois paramètres de type `struct point` et renvoie un résultat de type `struct triangle` (type introduit à l'exercice 8.1) représentant un triangle dont les trois sommets sont les paramètres.

## Chapitre 9

# Complexités

**Exercice de cours 9.1.** Soit une fonction  $f : \mathbb{N} \rightarrow ]1, +\infty[$ . Montrer que  $f = \mathcal{O}(f)$ .

**Exercice de cours 9.2.** Soit une fonction  $f : \mathbb{N} \rightarrow ]1, +\infty[$ . Montrer que  $f = \mathcal{O}(f^2)$ .

**Exercice de cours 9.3.** Trouver un exemple de fonction  $f : \mathbb{N} \rightarrow ]1, +\infty[$  tel que  $f \neq \Theta(n)$ .

## Chapitre 10

# Types algébriques ou comment fabriquer ses propres types en OCaml

## Chapitre 11

# Mutabilité en OCaml

## Chapitre 12

# Chaînes de caractères